

ServerlessLLM: 用于大语言模型的低延迟无服务器推理

Yao Fu¹ Leyang Xue¹ Yeqi Huang¹ Andrei-Octavian Brabete¹ Dmitrii Ustiugov² Yuvraj Patel¹ Luo Mai¹

¹University of Edinburgh ²NTU Singapore

摘要

本文介绍了 ServerlessLLM，一个分布式系统，旨在支持面向大语言模型（LLMs）的低延迟无服务器推理。通过利用推理服务器中接近 GPU 的存储和内存容量，ServerlessLLM 实现了有效的本地检查点存储，减少了对远程检查点下载的需求，并确保高效的检查点加载。ServerlessLLM 的设计有三个核心贡献：(i) 快速多级检查点加载，具有新的加载优化检查点格式和多级加载系统，充分利用了 GPU 服务器上复杂存储层次结构的带宽；(ii) 高效的 LLM 推理实时迁移，使新启动的推理能够利用本地检查点存储，同时确保最小的用户中断；(iii) 启动时间优化的模型调度，评估每个服务器上的检查点的局部性状态，并将模型调度到能最小化推理开始时间的服务器上。全面的评估，包括微基准测试和真实场景，表明 ServerlessLLM 在各种 LLM 推理工作负载下大幅优于最先进的无服务器系统，降低了 10 到 200 倍的延迟。

1 引言

大型语言模型（LLMs）最近被集成到各种在线应用程序中，例如编程助手 [26]、搜索引擎 [21] 和会话机器人 [54]。这些应用程序通过将用户输入（如问题）分解为词元（例如单词）来处理它们。LLMs 以自回归的方式生成响应，根据输入词元和已经生成的词元的组合来预测下一个词元，直到遇到句子结束符（EoS），即停止生成。为了优化这一过程，LLMs 使用键值缓存来存储中间结果，从而减少冗余计算。

以规模化方式提供大型语言模型（LLMs）面临重大挑战，因为这需要大量的 GPU 资源，并且需要满足交

互式服务提出的严格低响应时间约束。此外，LLM 推理延迟不可预测，因为它取决于输出长度，而由于迭代词元生成 [27, 42, 81]，输出长度会显著变化。

为了实现低延迟，处理一个 LLM 请求通常需要多个 GPU 持续数秒到数分钟。在实践中，托管 LLMs 的服务提供商需要满足各种类型的开发者需求，这导致了显著的 GPU 消耗 [20]，并影响了 LLM 服务的可持续性 [23]。因此，LLM 推理服务被迫对用户发送的请求数量施加严格限制（例如，ChatGPT 每 3 小时内最多 40 条消息 [54]），这凸显了服务提供商目前在满足需求方面的挑战。研究人员预测，随着 LLM 推理接近 Google 搜索的普及程度，其推理成本可能增加超过 50 倍 [23]。

为了减少 GPU 消耗，LLM 服务提供商正在转向无服务器推理，如 Amazon SageMaker [63]、Azure [50]、KServe [16] 和 HuggingFace [35] 等平台所示。在这种模型中，开发人员将包含模型执行和参数文件的 LLM 检查点上传到检查点存储系统。当接收到请求时，模型加载调度程序选择可用的 GPU 来启动这些检查点。然后请求路由器将推理请求导向选定的 GPU。这种无服务器方法使基础设施提供商能够高效地在 GPU 上复用 LLMs，提高资源利用率。此外，它还为基础设施用户提供了经济优势，用户只需为每次请求的持续时间付费，从而避免了昂贵的长期 GPU 预订费用。

尽管无服务器推理为部署大型语言模型（LLMs）提供了成本节约，但它也引入了显著的延迟开销。这些开销通常归因于推理冷启动，这是无服务器工作负载中常见的问题，如公共踪迹所示 [64, 92]。对于 LLM 检查点而言，冷启动尤其漫长，其大小可以从千兆字节 [11, 76, 91] 到太字节 [29] 不等。如此巨大的规模是由于这些模型中包含的参数数量极其庞大，导致从远程存储下载时会有

明显的延迟。此外，这些检查点由众多具有独特结构和大小的张量组成。将这些张量加载到 GPU 上的复杂过程，包括文件反序列化、内存分配和张量形状解析，进一步加剧了这些延迟。

我们旨在探索支持低延迟无服务器推理的系统设计，特别是针对大规模语言模型 (LLMs)。我们注意到，基于 GPU 的推理服务器通常具有一个复杂但利用率较低的存储层次结构，配备了大量主机内存和存储容量。当前的无服务器推理系统，如 KServe [16] 和 Ray Serve [73]，往往仅利用可用主机内存的一小部分，并且仅在有限程度上使用 SSD 来缓存来自模型仓库的检查点。这一观察促使我们提出了一种新颖的系统设计：利用多层存储层次结构进行本地检查点存储，并利用其显著的存储带宽以实现高效加载检查点。

然而，在实现本地检查点存储时，出现了几个悬而未决的问题：(i) 鉴于 GPU 服务器复杂的存储架构，包括多个 GPU、DRAM、SSD 和远程存储，并通过 PCIe、NVMe 和网络连接等多种链路相互连接，我们如何优化 LLM 检查点的加载以充分利用可用带宽？(ii) 将请求分配给已预加载检查点的服务器可以避免远程检查点下载的需求，但这种策略可能会导致长时间的排队延迟或高中断成本。这尤其具有挑战性，因为 LLMs 通常具有长时间且不可预测的推理持续时间，与传统的深度神经网络推理显著不同。(iii) 在分布式集群中，当并发处理模型请求并将检查点预先加载到本地存储的不同层次时，应选择哪些服务器以最小化启动模型推理所需的时间？

为了解决上述问题，我们设计并实现了 ServerlessLLM，它包含三个核心贡献：

(1) **快速多层检查点加载**。ServerlessLLM 可以最大化 GPU 服务器在 LLM 检查点加载时的存储带宽利用率。它引入了 (i) 一种新的 优化加载检查点，支持基于顺序、块状读取和高效的张量内存寻址，以及 (ii) 一种 高效的多层检查点存储系统，可以通过内存中的数据块池、内存复制高效的数据路径和一个多阶段数据加载流水线来利用多层存储层次结构上的大量容量和带宽。

(2) **高效的 LLM 推理实时迁移**。我们强调了 LLM 推理实时迁移的需求，并首次在无服务器推理系统中实现了 LLM 的实时迁移，以增强支持基于局部性驱动推理时的性能。为了在迁移 LLM 推理时实现高效率，我们实施了两种策略设计：(i) 源服务器仅迁移词元，而不是大型 kv 缓存，这大大减少了迁移过程中的网络流量；(ii) 它

在目标服务器上触发 kv 缓存的有效重新计算，确保迁移能够及时完成。

(3) **启动时间优化的模型调度**。ServerlessLLM 通过启用保时、感知局部性的模型调度来辅助无服务器推理系统。它集成了成本模型，以准确估计从存储层次结构的不同层级加载检查点的时间以及将 LLM 推理迁移到另一台服务器所需的时间。基于估算结果，Phantom 可以选择最佳服务器以最小化模型启动延迟。

我们已在 GPU 集群中针对各种基准方法对 ServerlessLLM 进行了全面评估。微基准测试结果表明，ServerlessLLM 的 LLM 检查点加载显著优于现有系统，如 Safetensors [36] 和 PyTorch [58]，加载速度提高了 3.6 到 8.2 倍。这种性能提升在 OPT [91]、LLaMA-2 [76] 和 Falcon [11] 等大型 LLM 中尤为明显。ServerlessLLM 还支持新兴的 LoRA 适配器 [34]，在检查点加载上实现了 4.4 倍的速度提升。

此外，我们使用基于公开的 Azure Trace [64] 建模的实际无服务器工作负载对 ServerlessLLM 进行了评估，并将其与 KServe、Ray Serve 以及带有本地检查点缓存的 Ray Serve 变体进行了基准测试。在这些场景中，ServerlessLLM 在跨数据集（即 GSM8K [25] 和 ShareGPT [83]）运行 OPT 模型推理时，延迟提高了 10 到 200 倍。这些结果强调了 ServerlessLLM 在结合快速检查点加载、高效推理迁移和优化模型加载调度方面的有效性。ServerlessLLM 的源代码在 <https://github.com/ServerlessLLM/ServerlessLLM> 发布。

2 背景与动机

2.1 为什么大型语言模型需要无服务器推理

众多公司，包括 Amazon、Azure、Google、HuggingFace、Together AI [6]、Deepinfra [2]、Replicate [5]、Databricks [7]、Fireworks-AI [3] 和 Cohere [4]，都推出了无服务器推理服务（也称为无服务器模型入口）。这些服务使用户能够以原样部署标准开源 LLM，或者通过微调对其进行修改，或运行自定义构建的模型。

通过仅按推理持续时间和处理数据量收费，无服务器推理可以显著降低 LLM 用户的成本。这些无服务器平台还提供诸如自动缩放和自动故障恢复等功能，以保持实例处于“始终在线”状态。对于提供公司而言，无服务器推理允许在 GPU 集群内有效地复用模型，提高

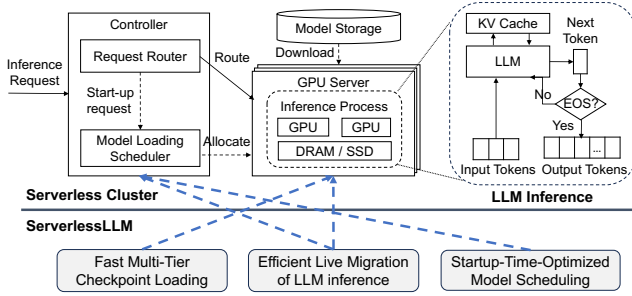


图 1: 概述 ServerlessLLM 引入的 GPU 无服务器簇、LLM 推理以及新设计。

资源利用率，并为用户管理基础设施生成软件溢价。

对于具有动态和不可预测工作负载的 LLM 应用，无服务器推理系统尤其有利。这些可能包括刚刚推出且用户参与度没有明确预测的新产品（例如 ChatGPT 服务的推出），或者面临自发和不可预测需求的应用，这在医疗、教育、法律和销售等领域很常见。与全球规模的 LLM 服务不同，这些应用仅在用户访问 LLM 服务时被激活。

2.2 无服务器集群与大模型推理

我们介绍了图 1 中在 GPU 无服务器集群中的关键组件。当接收到新的推理请求时，控制器将其分发到运行 LLM 推理服务实例的集群中的 GPU 装备结点，并发送到托管模型检查点的云存储。控制器通常包括两个主要组件：请求路由器和模型加载调度器。请求路由器将传入请求导向已运行 LLM 推理进程的结点，或者指示模型加载调度器在未分配的 GPU 上激活 LLM 推理进程。所选 GPU 结点启动一个 GPU 进程/容器，设置一个推理库（例如 HuggingFace Accelerate [32] 和 vLLM [42]）。此推理过程涉及从远程模型存储下载所需模型的检查点并加载到 GPU，通过 SSD 和 DRAM。

LLM 推理过程通常处理包含用户指定输入提示的请求，即一系列词元，如图 1 所示。此过程基于提示和之前生成的所有词元迭代地生成词元，直到产生一个句子结束符（记为 EoS），导致非确定性的总推理时间 [55]。在每次迭代中，LLM 在 KV 缓存中缓存中间计算结果以加速后续词元的生成 [42, 56]。每次迭代生成的词元会被持续流式传输回请求客户端，使 LLM 应用本质上具有交互性。因此，其性能通过首词元延迟（即返回第一个词元所需的时间）和每词元延迟（即生成一个词元的

平均时间）来衡量。

2.3 Serverless 部署大模型推理的挑战

在无服务器系统上部署大型语言模型（LLMs），虽然前景广阔，但常常伴随着显著的延迟开销。这主要是由于无服务器集群中冷启动的比例较高，如公共数据显示：Azure 的 Trace [64] 表明，在 5 分钟的保持活动间隔内，超过 40% 的功能显示冷启动率超过 25%，大约 25% 的功能经历超过 60% 的冷启动率。这些数据与我们的实验结果一致，突显了实际场景中冷启动的影响。因此，包括彭博社在内的许多无服务器提供商都公开承认，在其平台上初始化最先进的 LLMs 进行推理时，经常遇到高达数十秒的极高延迟。

我们观察到长 LLM 冷启动延迟的几个主要原因：

(1) **大型语言模型检查点体积庞大，延长了下载时间。** 大型语言模型检查点的体积显著大于传统的深度神经网络检查点，这导致了更长的下载时间。例如，Grok-1 [82] 检查点超过 600 GB，DBRX [72] 是 250 GB，而 Mixtral-8x22B [71] 约为 280 GB。¹ 下载如此大的检查点从远程存储变得昂贵。例如，从 S3 或 Blob 存储获取一个大小为 130GB 的 LLM 检查点（如 LLaMA-2-70B [76]），使用带宽为 5GB/s 的商品网络至少需要 26 秒 [19]。

(2) **加载 LLM 检查点是一个冗长的过程。** 即使模型检查点存储在本地的 NVMe SSD 上，将这些检查点加载到 GPU 中仍然是一个复杂的过程，包括模型初始化、GPU 内存分配、张量创建和张量数据复制，通常需要数十秒（如 §7.2 节所述）。例如，使用 PyTorch 将 OPT-30B 模型加载到 4 个 GPU 中需要 34 秒，而将 LLaMA-2-70B 加载到 8 个 GPU 中需要 84 秒。这种加载延迟远远超过推理过程中生成一个 Token 所需的时间，后者通常小于 100ms [55]。因此，较长的第一个 Token 延迟会显著影响用户体验。

2.4 现有解决方案及相关问题

为了改善支持大型语言模型（LLMs）时的延迟性能，现有的解决方案暴露出各种各样的问题：

(1) **超订阅 GPU。** 普遍的解决方案 [13, 84]，旨在绕过无服务器推理集群中的模型下载和加载时间，通常涉及超订阅 GPU 以适应峰值需求场景。例如，AWS 无服务器

¹以浮点数 16 位 (float16) 精度计算的模型大小。

推理 [13] 维持一定数量的处于预热状态的 GPU 实例，以减轻冷启动缓慢的影响。虽然此策略对于管理常规较小的模型（如 ResNet 和 BERT）是有效的，但对于需要大量资源的 LLM 来说却具有挑战性，因为这些模型需要来自昂贵 GPU 的更多资源。

(2) **在主机内存中缓存检查点**。已开发出几种解决方案 [33, 39]，这些方案在 GPU 服务器的主机内存中缓存模型检查点，以消除下载模型的需求。这种方法通常对较小的传统模型（例如，多达几 GB [39]）有效。然而，仅依靠基于主机内存的缓存对于 LLMs 来说是不够的。LLMs 的大小很容易超过数百 GB，这挑战了主机内存存储足够数量的它们的检查点的能力。主机内存的有限容量导致显著的缓存缺失，从而引起频繁的模式下载，如 §7.4 进一步讨论的那样。

(3) **部署额外的存储服务器**。各种策略 [19] 推荐在本地簇内部署额外的存储服务器以缓存模型检查点。尽管有这些改进，最近的跟踪研究 [19] 表明，即使连接到配备 100 Gbps 网卡的本地商品存储服务器，通过优化的流水线下载模型仍可能超过 20 秒。虽然引入更快的网络（例如，200 Gbps 以太网或 InfiniBand）可以减少此延迟，但实施额外存储服务器和高带宽网络的成本仍然很高 [18, 31]。例如，使用网络优化的 AWS ElasticCache 服务器 [1] 来支持一个 70B 模型可能导致成本增加 100%。具体来说，提供 210GB 内存和 200 Gbps 网络性能的 cache.c7gn.16xlarge 服务器价格为每小时 \$16.3，相当于一个 8-GPU g5.48xlarge 服务器的成本。

3 利用服务器内多层存储

ServerlessLLM 采用了一种在成本、可扩展性以及长期可行性方面均表现优异的设计方法，解决了前几节中强调的挑战，即高模型下载时间和漫长的模型加载时间。

3.1 设计直觉

我们的设计受到一个简单观察的启发，即用于推理的 GPU 服务器具有多级存储层次结构，具有较大的容量和带宽。从容量的角度来看，这些服务器配备了广泛的内存能力。例如，一台当代 8-GPU 服务器可以支持多达 4TB 的主内存，64TB 在 NVMe SSD 上，192TB 在 SATA SSD 上 [52]。此外，我们观察到在无服务器推理环境中，GPU 服务器中的大部分主机内存和存储设备未

被充分利用。

在带宽方面，GPU 服务器通常配备多个 GPU，每个 GPU 通过专用的 PCIe 连接与主机内存相连，从而在内存和 GPU 之间提供显著的聚合带宽。NVMe 和 SATA 固态硬盘也通过各自的链路连接，并可以通过 RAID 配置来提高吞吐量。例如，使用 PCIe 5.0 技术的 8-GPU 服务器可以在主机内存和 GPU 之间实现高达 512 GB/s 的聚合带宽，而 NVMe 固态硬盘（RAID 0）到主机内存的带宽约为 60 GB/s。

基于这些观察，我们提出了一种设计方法，利用服务器内未使用的多层存储容量来本地存储模型并更快地加载它们，从而减少延迟。这种方法 (i) 成本效益高，因为它重新利用了 GPU 服务器中现有的、未充分利用的存储资源；(ii) 可扩展，因为可用的本地存储容量和带宽可以随着更多推理服务器的添加而自然增加；并且 (iii) 长期可行，因为未来的 GPU 服务器将包含更大的容量和带宽（例如，每个 Grace-Hopper GPU 配备了 1 TB 片上 DRAM 以及 900GB/s 的片上 DRAM 与 HBM 之间的 C2C 链路）。

3.2 设计关注点与概述

在实施我们的设计时，我们确定了必须解决的三个关键问题。

(1) **支持复杂的多层存储层次结构**。当前的检查点和模型加载工具，如 PyTorch [58]、TensorFlow [75] 和 ONNX Runtime [62]，主要设计用于增强模型开发的训练和调试阶段。然而，这些工具并未针对读取性能进行优化，在无服务器推理环境中这一点变得至关重要。在这些情景中，模型检查点仅存储一次，但需要频繁地跨多个 GPU 加载和访问。这种对读操作的不足优化导致了显著的加载延迟。尽管像 Safetensors [36] 这样的解决方案可以提升加载性能（如第 7 节所示），但它们仍然未能充分利用多层存储层次结构的能力。

(2) **强局部性驱动的推理**。仅支持高效的模型加载是不够的；我们还需要能够有效调度请求到具有本地存储检查点的 GPU 服务器的方法。然而，实现基于局部性的大语言模型（LLM）推理带来了挑战。当前的机器学习模型服务系统，如 ClockWork [33] 和 Shepherd [90]，考虑了检查点的局部性。然而，它们要么依赖于对模型推理时间的准确预测（这对于 LLM 来说存在问题），要么中断正在进行的模型推理，导致显著的停机时间和冗余

计算。因此，ServerlessLLM 必须采用一种针对 LLM 推理独特特性的新方法（即此工作负载是交互式的，并且具有长且不可预测的持续时间），这需要支持推理的动态迁移，详见第 5 节。

(3) **优化启动时间的调度模型。** ServerlessLLM 旨在最小化模型启动延迟。集群调度器（或控制器）在将模型调度到 GPU 资源以响应传入推理请求方面起着至关重要的作用。然而，调度器需要仔细考虑检查点在整个集群中的局部性。许多因素可能会影响整体启动延迟，例如内存层次结构中每一层提供的带宽差异。有时，将当前推理执行转移到新的 GPU 比将请求分配给必须从存储介质加载模型的 GPU 更有利。因此，ServerlessLLM 需要准确估计启动时间，考虑到集群的检查点局部性状态，并相应地分配资源以最小化启动时间。

概述。 ServerlessLLM 通过三项新颖的设计解决了这些问题，如图 1 所示。首先，它促进了快速的多级检查点加载（第 4 节），以充分利用每个 GPU 服务器的存储容量和带宽。它还协调 GPU 服务器和集群控制器，以实现高效的 LLM 推理实时迁移（第 5 节），确保以最小的资源开销和用户干扰实现基于局部性的推理。最后，ServerlessLLM 的控制器具有一种启动时间优化的模型调度策略（第 6 节），有效分析集群中每个服务器的检查点存储状态，并选择一个服务器来启动模型，从而将启动时间降至最低。

4 快速多层检查点加载

在本节中，我们介绍 ServerlessLLM 中快速多层检查点加载的设计，有若干关键目标：(i) 充分利用 GPU 服务器上多层本地存储的带宽和容量，(ii) 确保可预测的加载性能，这对 ServerlessLLM 在低延迟推理集群中的就绪状态至关重要，(iii) 维持一种支持来自各种深度学习框架检查点的通用设计。

4.1 加载优化检查点

我们的设计灵感来源于这样一个观察：LLM 模型检查点在训练和调试过程中经常被写入，但很少被加载。相反，在无服务器推理环境中，检查点只上传一次，但会被多次加载。这种差异促使我们将其转换为以加载优化的格式。

为了确保我们的设计对不同框架具有通用性，我们

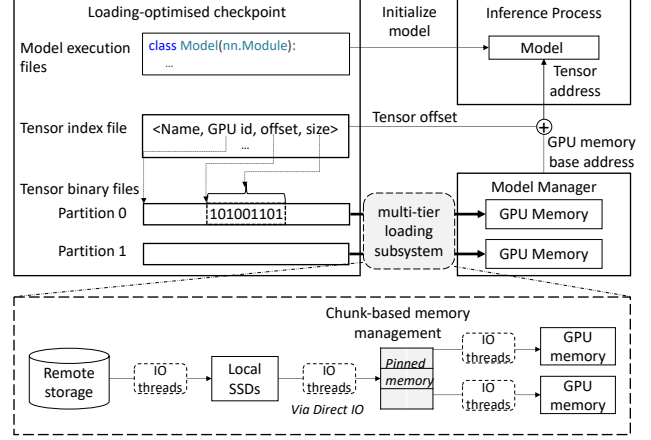


图 2: 快速多层检查点加载中的组件。

在检查点的通用假设下进行操作。检查点包含：(i) 模型执行文件，它定义了模型架构。根据框架的不同，格式也有所不同；TensorFlow 通常使用 protobuf 文件 [74]，而 PyTorch 使用 Python 脚本 [57]。除了架构之外，这些文件还详细说明每个张量的大小和形状，并包含一个模型并行计划。该计划在检查点加载时指定每个张量的目标 GPU。(ii) 模型参数文件，用于存储 LLM 中参数的二进制数据。这些文件中的张量可以以任意顺序排列。运行时（如 PyTorch）还可以将张量形状存储为索引来计算每个张量的偏移量和大小。

为确保快速加载性能，我们为转换后的检查点实现了两个主要特征：(i) 基于序列块的读取：为了确保高效的顺序读取，每个 GPU 上的张量被分组到划分中（如图 2 所示）。这些文件仅包含模型参数的二进制数据，并排除了元数据（如张量形状），便于大块读取。(ii) 直接张量寻址：我们创建了一个张量索引文件（如图 2 所示），该文件将张量名称映射到一个包含 GPU ID、偏移量和大小的元组，从而促进张量的高效恢复。张量与内存字大小对齐，便于直接计算内存地址。

我们观察到，将加载和推理过程解耦可以进一步提升加载性能。这种分离允许检查点的加载被预先调度并与推理过程的初始化重叠进行。为此，ServerlessLLM 使用一个模型管理器来加载张量数据，同时允许推理过程专注于通过为每个张量设置数据指针来初始化模型。更具体地说，模型管理器在 GPU 上分配内存并通过快速多级加载子系统加载检查点的二进制数据（详见 4.2）。推理过程初始化模型对象并为每个张量设置 GPU 内存地址。它从模型管理器获取每个 GPU 的基地址（即 CUDA

IPC 句柄) 并从张量索引文件中读取张量偏移量, 从而计算出张量 GPU 内存地址 (即 $\text{base} + \text{offset}$)。为了确保模型在推理开始前完全初始化, 推理过程和模型管理器执行同步操作。

4.2 多级加载子系统

为了实现快速且可预测的检查点加载性能, 我们设计了一个多级加载子系统, 集成在模型管理器中。该子系统包含了几种技术:

基于块的数据管理。 为了实现快速加载性能, 我们实现了具有三个主要特征的基于块的数据管理: (i) 利用并行 PCIe 链路。为了解决从存储加载多个模型到 GPU 时由单一 PCIe 链路引起的瓶颈问题, 我们采用并行 DRAM-to-GPU PCIe 链路以促进跨 GPU 的同时检查点加载。(ii) 支持应用特定控制。我们的内存池通过提供用于分配和释放内存的 API, 超越了简单的缓存功能。这使得可以根据应用程序的具体需求对缓存或驱逐的数据块进行细粒度管理。(iii) 缓解内存碎片化。我们通过使用固定大小的内存块来解决由内存碎片化引起的延迟和空间效率低下的问题。

可预测的数据路径。 我们在模型管理器中创建了一个高效的数据路径, 采用了两种主要策略: (i) 利用直接文件访问。我们使用直接文件访问 (例如 Linux 中的 ‘O_DIRECT’) 来避免过多的数据复制, 通过直接将数据读取到用户空间来实现。这种方法优于内存映射文件 (mmap), 目前在 Safetensors [36] 等高速加载器中使用, 后者依赖系统缓存且缺乏一致的性能保证 (这对可预测性能至关重要)。(ii) 利用固定内存。我们利用固定内存来消除 DRAM 和 GPU 之间的冗余数据复制。此方法允许直接将数据复制到 GPU, 仅需少量 CPU 参与, 确保以单线程方式高效利用 PCIe 带宽。

多层加载流水线。 我们开发了一种多层加载流水线, 以支持各种存储接口并提高加载吞吐量。该流水线具有以下三个特征: (i) 对多种存储接口的支持。ServerlessLLM 为各种存储接口提供了专用的函数调用, 包括本地存储 (例如, NVMe、SATA)、远程存储 (例如, S3 对象存储 [12]) 和内存存储 (固定内存)。它在每种情况下都使用适当的方法实现高效的数据访问。(ii) 支持同一层内的并行性。为了利用现代存储设备的高并发能力, ServerlessLLM 在每个存储层中使用多个 I/O 线程来读取数据, 从而提高带宽利用率。(iii) 灵活的流水线结构。我

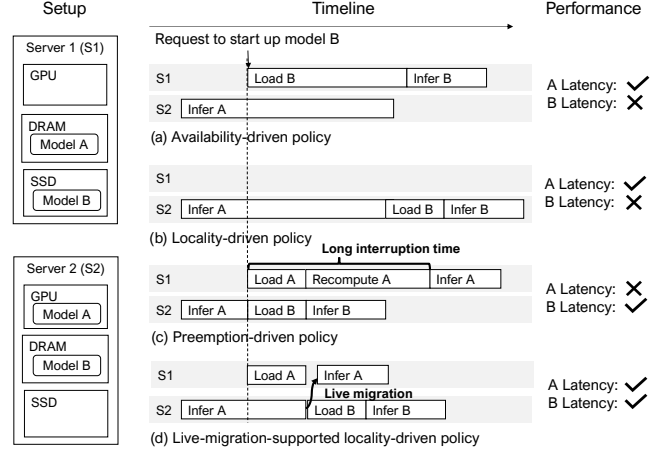


图 3: 不同地方性政策的分析

们采用基于任务队列的流水线设计, 支持新存储层的高效集成。I/O 线程读取存储块并将它们的索引 (偏移量和大小) 入队到下一存储层的 I/O 线程中。

5 高效的大语言模型推理迁移

在本节中, 我们将描述为什么实时迁移是有效的位置驱动 LLM 推理的关键, 并说明如何使这种实时迁移过程特别高效。

5.1 需要实时迁移

我们考虑一个简单示例来分析当前不同方法在支持检查点局部性方面的性能。在这个示例中, 我们有两个服务器 (名为 Server 1 和 Server 2) 以及两个模型 (名为 Model A 和 Model B), 如图 3 所示。Server 1 当前在 DRAM 中存储了 Model A, 在 SSD 中存储了 Model B, 并且其 GPU 处于空闲状态, 而 Server 2 当前在 DRAM 中存储了 Model B, 并且其 GPU 正在运行 Model A 的推理任务。

在图 3 中, 我们分析了启动模型 B 的潜在策略的性能。我们的分析基于它们对模型 A 和模型 B 的延迟性能的影响:

- 可用性驱动的策略当前选择拥有可用 GPU 的 Server 1, 且对 Model B 的位置不敏感。因此, Model B 的启动延迟受到影响, 而 Model A 不受影响。
- 以 locality 为导向的策略选择服务器时考虑 locality, 因此在服务器 2 上启动模型 B。然而, 它会等待模型

A 完成，导致模型 B 遭受长时间的排队延迟。此外，locality 策略使服务器 1 的利用率低下，无法充分利用所有服务器。

- 抢占驱动的策略在服务器 2 上抢占模型 A 并启动模型 B。它识别到服务器 1 是空闲的，并在那里重新启动模型 A。此策略减少了模型 B 的延迟，但在模型 A 执行重新加载和重新计算时导致了显著的停机时间。
- 支持实时迁移的以 locality 驱动的策略在不干扰 Model A 的情况下优先考虑 locality。它最初在 Server 1 上预加载 Model A，保持推理操作继续进行。当 Model A 设置在 Server 1 上时，将其中间状态转移到该服务器，无缝继续推理。随后，Model B 在 Server 2 上启动，利用 locality。此策略优化了 Model A 和 Model B 的延迟。

根据上述示例，活迁移在所有基于局部性的策略中，在改善模型 A 和模型 B 的延迟方面表现出色。

5.2 使云迁移高效进行

我们旨在实现高效的 LLM 推理在线迁移，同时尽量减少资源开销和用户中断。我们最初考虑使用 Singularity [68] 的快照方法，该方法涉及对 LLM 推理进行快照。然而，由于快照创建和传输时间较长（例如通常需要 10 秒甚至几分钟），此方法速度较慢。基于脏页的迁移可能被考虑以加速虚拟机迁移，但这种方法目前在支持 GPU 的容器和虚拟机中不受支持。因此，我们决定探索可以在应用程序中轻松实现的在线迁移方法。

为了使实时迁移方法在大规模语言模型（LLM）推理中有效，我们的目标是实现两个目标：(i) 迁移的推理状态必须是最小的，以减少网络流量，(ii) 目标服务器必须快速与源服务器的进度同步，以尽量减少迁移时间。

对于 (i)，我们建议迁移词元（通常为 10-100 KB）而不是大型 KV 缓存（通常为 1-10 GB），因为在目标 GPU 上基于迁移的词元重新计算 KV 缓存通常比通过网络传输脏状态要快得多。在某些情况下（例如，给定高带宽网络和短输入序列），迁移 KV 缓存也可能很快，但它仍然会增加集群网络流量，而迁移词元则不会。

对于 (ii)，我们借鉴了 LLM 推理中的一个见解：在目标 GPU 上重新计算当前词元的 KV 缓存明显更快（通常快一个数量级）比在源 GPU 上生成相同数量的新词元。这种方法促进了基于词元的多轮迁移的有效收敛，

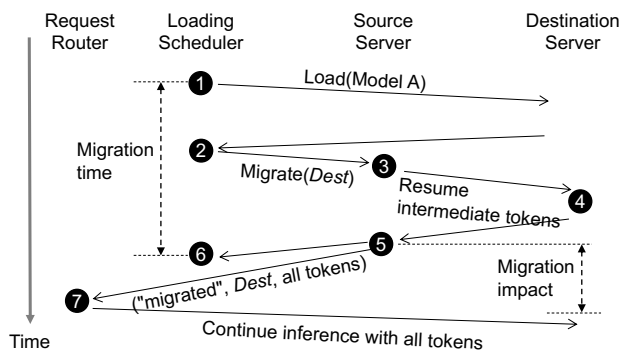


图 4: LLM 推理的实时迁移过程

随着每一轮的进行，源 GPU 上生成的词元数量逐渐减少。例如，根据 [70]，重新计算 1000 个词元的 KV 缓存所需的时间相当于生成大约 100 个新词元所需的时间。

5.3 多轮实时迁移过程

我们实现上述提议作为一个多轮实时迁移过程。在每次迁移轮次（步骤 ③、④ 和 ⑤）中，目标服务器（称为 dest 服务器）使用源服务器（称为 src 服务器）发送的中间词元重新计算 KV 缓存。当源服务器和目标服务器之间的差距（即最后一轮后生成的词元）足够小时，src 服务器停止生成，并通过请求路由器将所有词元发送到 dest，确保迁移过程中正在进行的推理中断最小化。此迁移过程如图 4 所示，其步骤定义如下：

1. 模型加载调度器向 dest 服务器发送模型加载请求以将模型 A 加载到 GPU 中。如果在 dest 服务器上存在模型 A 的空闲实例，调度器跳过此步骤。
2. 加载完成后，调度器向 src 服务器发送携带 dest 服务器地址的迁移请求。
3. 收到迁移请求后，src 服务器将其自身设置为“正在迁移”，如果推理未完成，则向 dest 服务器发送带有中间词元（即输入词元和第 3 步之前产生的输出词元）的 resume 请求；否则，它立即返回调度器。
4. 目标服务器根据恢复请求中的词元重新计算 KV 缓存。
5. 一旦完成 resume 请求，src 服务器停止推理，返回调度器，并使用所有标记（即步骤 3 到步骤 5 之间生成的中间标记和剩余标记）以及标志“migrated”回复请求路由器。

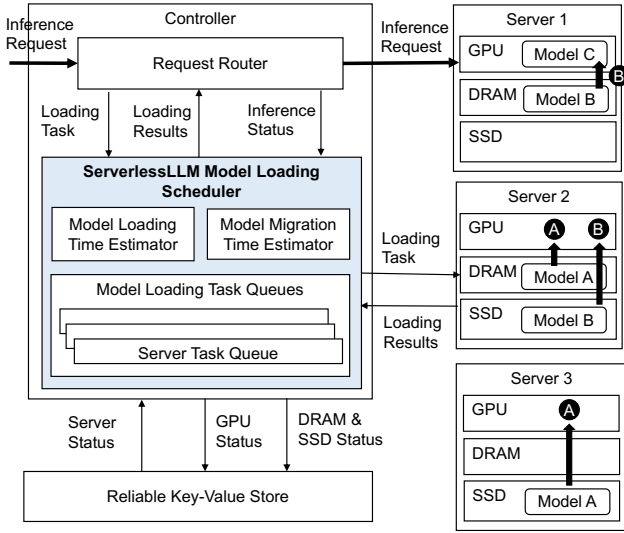


图 5: 模型加载调度器设计概述

- 调度器完成迁移，在 src 服务器卸载模型 A 并开始加载模型 B。
- 请求路由器检查推理响应中的标志。如果它是“迁移中”，请求路由器在其路由表中用 dest 服务器替换 src 服务器，并将所有 Token 发送到 dest 服务器以继续推理。

5.4 实际考虑

处理推理完成。 LLM 推理的自回归特性可能导致任务在 src 服务器的步骤③和④之间完成。在这种情况下，src 服务器像往常一样通知请求路由器推理已完成，并且还通知加载调度器，后者随后指示 dest 服务器停止恢复，终止迁移过程。

处理服务器故障。 ServerlessLLM 可以在 LLM 推理迁移过程中管理服务器故障。在 src 服务器发生故障的场景中，如果故障发生在加载期间（即图 4 中的步骤②之前），调度器将中止迁移并将模型从目标服务器卸载。如果故障发生在迁移期间（即步骤②和③之间），调度器会指示目标服务器清除任何恢复的 KV 缓存并卸载模型。

在 dest 服务器失败的情况下，如果失败发生在加载期间，调度器将取消迁移。如果失败发生在恢复期间，源服务器会通知调度器失败并继续进行推理。

6 启动时间优化的模型调度

在本节中，我们描述了在 ServerlessLLM 的集群调度器（记为控制器）中实现的启动时间优化的模型调度设计，如图 5 所示。该调度器处理来自请求路由器的加载任务，并采用两个关键组件：模型加载时间估计量和模型迁移时间估计量。前者评估从各种存储介质加载的时间，而后者估算必要的模型迁移时间。例如，如图 5 所示，调度器计算从不同服务器的 DRAM 和 SSD 加载模型 A（标记为Ⓐ）所需的时间，以辅助服务器选择。类似地，对于模型 B（Ⓑ），它评估是否将模型 C 迁移到另一台服务器或从服务器 2 的 SSD 加载模型 B。

为确保时间估计的鲁棒性，ServerlessLLM 调度程序为每台服务器使用独立的加载任务队列，有效减轻了并发加载活动引起的争用影响。在分配任务时，它会及时更新服务器状态（包括 GPU 和 DRAM/SSD 状态）到一个可靠的键-值数据库（如 etcd [30] 和 ZooKeeper [38]）。此机制使 ServerlessLLM 能够保持连续性并高效地从故障中恢复。

6.1 估计模型加载时间

为估计从不同存储层加载模型所需的时间，我们考虑三个主要因素：(i) 排队时间 (q)，即模型在服务器加载任务队列中的等待时间。当其他模型正在同一服务器上等待加载时就会发生这种情况；(ii) 模型大小 (n)，以字节为单位表示的模型大小，或多 GPU 推理场景中的模型划分；(iii) 带宽 (b)，即从存储传输到 GPU 的可用速度。ServerlessLLM 跟踪网络、SSD 和 DRAM 的带宽，使我们可以将加载时间计算为 $q+n/b$ 。这里， q 累积自对已进入队列的模型的先前估计。

对于精确估计，我们已经实现了以下几点：(i) 每台服务器按顺序加载模型，为远程 SSD 和 SSD-DRAM 路径使用单一的 I/O 队列（因为这些路径被服务器上的多个 GPU 共享），减少了带宽竞争，这会复杂化估计；(ii) 在多层存储中，ServerlessLLM 使用最慢的带宽进行估计，由于 ServerlessLLM 的流水线加载设计。例如，当 SSD 和 DRAM 都参与时，SSD 带宽是关键瓶颈，因为它比 DRAM 慢几个数量级；(iii) 调度器监控服务器返回的加载延迟。它利用监控指标不断改进对通过不同存储介质的带宽估计。

6.2 估算模型迁移时间

对于在线迁移时间的估计，我们的重点是模型恢复时间（如图 4 中步骤 ④ 所示），因为这比通过网络传输词元（毫秒级）要慢得多（秒级）。我们计算模型恢复时间时考虑以下因素：(i) 输入词元 (t_{in})，即 LLM 输入提示中的词元数量；(ii) 输出词元 (t_{out})，即已经生成的词元；以及 (iii) 模型特定参数 (a 和 b)，这些参数会因每个 LLM 的批量大小和其他因素而变化，基于像 vLLM [42] 这样的 LLM 系统研究。结合上述所有因素，我们可以计算模型恢复时间为 $a \times (t_{in} + t_{out}) + b$ 。

然而，从服务器获取实时输出词元以供调度器使用可能会由于过多的服务器交互而导致瓶颈。为了解决这个问题，我们开发了一种方法，调度器向本地请求路由器查询某个模型的推理状态，如图 5 所示。通过推理时长 (d) 和生成一个词元的平均时间 (t)，我们计算出 $t_{out} = d/t$ 。

对于选择最佳服务器以进行模型迁移，ServerlessLLM 使用动态规划方法以最小化迁移时间。

6.3 实际考虑

选择最佳服务器。 利用我们的时长估计，ServerlessLLM 评估所有服务器以加载即将到来的模型，并选择估计启动时间最短的服务器。选择结果包括服务器 ID 和要分配的 GPU 插槽数。如果没有可用的 GPU，即使在考虑迁移的情况下也是如此，则加载任务将被挂起并等待，一旦请求路由器通知调度器释放 GPU 后重试。

处理调度器故障。 ServerlessLLM 构建为能够承受故障，利用一个可靠的键-值数据库来跟踪服务器状态。在接受到服务器加载任务时，其 GPU 状态会立即更新在这个数据库中。服务器确认任务完成之后，调度器会在数据库中更新服务器的存储状态。一旦记录完成，调度器会通知请求路由器任务已完成，从而允许请求路由到该服务器。在调度器发生故障的情况下，恢复涉及从键-值数据库中检索最新的服务器状态，并将其同步到所有服务器。

缩放调度器。 通过为服务器状态的读取、写入和估计实现异步操作，加载调度器的性能得到了显著提升。目前的基准测试表明，它能够在标准服务器上每秒处理数千个加载任务。其分布式缩放的计划已预留用于未来开发。

资源公平性。 ServerlessLLM 对所有模型赋予同等的重

要性，并确保迁移不会影响延迟。虽然我们目前在 I/O 路径上采用顺序模型加载，但在服务器上探索具有公平性保证的并发加载将是未来的工作。

估计量的准确率。 我们的估计量可以根据服务器返回的监控指标持续改进其估算，如第 7 节所示，它们对服务器选择提供了足够的准确率。

7 评估

本节对 ServerlessLLM 进行全面评估，涵盖三个关键方面：(i) 评估我们优化加载的检查点和模型管理器的性能，(ii) 检查与 LLM 推理相关的实时迁移效率和开销，并 (iii) 基于真实服务器 less 跟踪数据的大规模 serverless 工作负载评估 ServerlessLLM。

7.1 评估设置

设置。 我们有两个测试平台：(i) 一台 GPU 服务器配备了 8 块 NVIDIA A5000 GPU，1TB DDR4 内存，2 颗 AMD EPYC 7453 CPU，两个 PCIe 4.0 兼容的 NVMe 4TB SSD (RAID 0 配置) 和两个 SATA 3.0 4TB SSD (RAID 0 配置)。该服务器通过 1 Gbps 网络连接到一个存储服务器，在其上部署了 MinIO [51]，一个兼容 S3 的对象存储；(ii) 一个 GPU 集群由 4 台服务器组成，这些服务器通过 10 Gbps 以太网互连。每台服务器配备了 4 块 A40 GPU，512 GB DDR4 内存，2 颗 Intel Xeon Silver 4314 CPU 和一块 PCIe 4.0 NVMe 2TB SSD。

模型。 我们使用最先进的大型语言模型 (LLMs)，包括不同大小的 OPT [91]、LLaMA-2 [76] 和 Falcon [11]。对于测试床 (ii) 上的聚类评估 (§7.3 和 §7.4)，遵循先前工作 [44]，我们分别为 OPT-6.7B、OPT-13B 和 OPT-30B 模型复制 32/16/8 个实例（除非另有说明），在评估过程中将它们视为不同的模型。

数据集。 我们使用真实世界的 LLM 数据集作为模型的输入。这包括包含人类问题编写者创建的问题的 GSM8K [25]，以及包含来自 GPT4 的多语言聊天的 ShareGPT [83]。由于我们使用的模型最多可以处理 2048 个上下文长度，我们将输入的词元数截断到最大长度。我们还从每个数据集中随机采样 4K 个样本，以创建混合工作负载，模拟真实的推理工作负载。

工作负载。 由于没有公开可用的 LLM 无服务器推理工

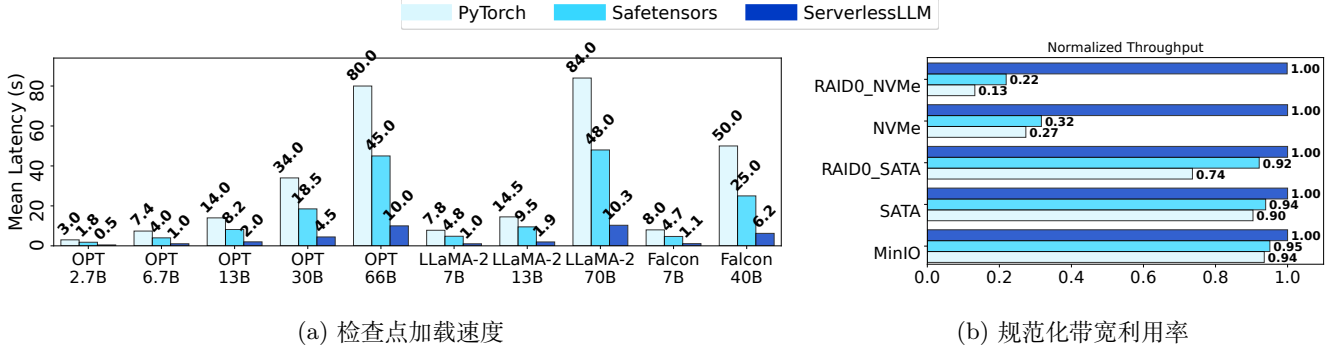


图 6: 检查点加载性能

作负载，我们使用 Azure 无服务器 [64] 跟踪，这是一个在最近的无服务器研究 [61] 和模型服务研究 [44, 90] 中使用的代表性无服务器工作负载。[44] 按照 AlpaServe 中使用的工作负载生成方法，将函数分配给模型，并创建具有高突发性的请求跟踪（使用 Gamma 分布，变异系数 $CV=8$ ）。然后我们将此跟踪扩展到所需的每秒请求数（RPS）。对于集群评估，我们根据模型的受欢迎程度复制每个模型，并使用轮询放置策略将其分布在节点的 SSD 上，直到达到整个集群的存储限制为止。检查点放置的优化被视为一个单独的问题，在本文中未加以解决。对于所有实验（除非另有说明），我们报告模型启动延迟，这是无服务器推理场景中的关键指标。当启用迁移或抢占时，此延迟会加上暂停延迟，以考虑延迟的影响。

7.2 ServerlessLLM 加载检查点

我们现在评估模型管理器在减少模型加载延迟方面的有效性。在我们的实验中，我们在测试环境 (i) 中测试检查点的读取。我们记录从每个模型检查点的 20 个副本的读取，以获得具有统计意义的性能报告。在完成检查点副本后，我们清除页面和 inode 缓存，以确保冷启动。对于每种类型的模型，我们随机访问这 20 个副本以模拟真实的访问模式。

加载性能。 我们旨在量化由 ServerlessLLM 检查点管理器实现的性能增益。我们比较了 PyTorch [58] 和 Safetensors [36]，分别代表按张量读取的检查点加载和基于 mmap 的检查点加载。我们在 FP16 下使用所有类型的模型和所有检查点，并在具有 12 GB/s 吞吐量的 RAID0-NVMe SSD 上运行测试。

图 6a 显示了所有模型在平均延迟方面的性能比

较²。我们观察到，对于我们的最小模型 (OPT-2.7B)，ServerlessLLM 比 PyTorch 和 Safetensors 分别快 6 倍和 3.6 倍。对于最大模型 (LLaMA-2-70B)，我们观察到类似的结果，其中 ServerlessLLM 分别比 PyTorch 和 Safetensors 快 8.2 倍和 4.7 倍。由于大量页错误（冷启动时为 112K 对于 LLaMA-2-7B），Safetensors 比 ServerlessLLM 慢。相比之下，ServerlessLLM 的检查点管理器利用直接 I/O 并实现基于块的并行加载，所有这些都显著提高了加载吞吐量。在我们的评估中，PyTorch 比 Safetensors 慢约 2 倍，这与 Safetensors 在一个公共 [37] 报告的基准测试中得出的结果一致。主要原因在于 PyTorch 首先将数据复制到主机内存，然后再复制到 GPU 内存。

此外，我们观察到 ServerlessLLM 的加载性能与模型类型无关。例如，OPT-13B 和 LLaMA-2-13B 的性能相似，表明性能仅依赖于检查点的大小。

LoRA 适配器的加载性能。 ServerlessLLM 同时支持以 PEFT 格式 [49] 加载 LoRA 适配器 [34]。我们在 [65] 相同的设置下进行了实验。对于 LLaMA-70B 模型的一个适配器 (Rank=32, Size=1GB)，ServerlessLLM 的加载延迟为 83.5ms，比 Safetensors 快 4.4 倍 (Safetensors 的加载延迟为 370ms)。这展示了 ServerlessLLM 在小检查点加载方面的加载器设计效率。

充分利用存储设备的完整带宽。 现在我们来了解 ServerlessLLM 是否能够利用存储介质提供的全部带宽以实现低延迟。我们使用上述描述的相同设置。我们选择 LLaMA-2-7B 来代表最先进的大型语言模型 (LLM)。我们使用 FIO [17]，配置为异步 4M 直接顺序读取，深度为 32 的基准设置，并通过所有存储介质中的优化吞

²模型名称后的数字代表图中参数的数量，B 代表十亿。

吐量结果来优化吞吐量。我们测试了 FIO 的各种设置，以确保所选配置在每种存储介质上具有最高的带宽。对于网络上的对象存储，我们使用官方 MinIO 基准测试来获取最大吞吐量。

图 6b 显示了不同存储设备的带宽利用率，归一化是使用 FIO 和 MinIO 获得的测量值。从下到上的存储设备在最大带宽上是递增的。我们观察到，ServerlessLLM 的模型管理器能够利用不同的存储介质并使其整个带宽饱和以获得最大性能。有趣的是，我们观察到与 Pytorch 和 Safetensors 相比，ServerlessLLM 更适合更快的存储设备（例如 RAID0-NVMe）。这表明现有的机制无法适应更新、更快的存储技术。尽管加载过程通过整个内存层次结构，ServerlessLLM 仍能够使带宽饱和，突显了对加载过程进行流水线处理的有效性。

性能分解。 现在我们着重说明模型管理器中的每一项最优化如何对整体性能做出贡献。我们使用 RAID0-NVMe 并采用不同的 OPT 模型运行实验。从基本实现（ReadByTensor）开始，逐步添加最优化直至 Pipeline 实现。图 7 显示了每个模型的性能分解。尽管不同模型的检查点大小不同，但我们观察到所有模型中各项最优化的贡献相似。

批量读取可将吞吐量提高 1.2 倍，缓解了逐个读取小张量时的吞吐量下降问题（模型中平均有三分之一的张量小于 1MB）。直接 IO 可将吞吐量提高 2.1 倍，绕过内核中的缓存和数据复制。多线程可将吞吐量提高 2.3 倍，因为 SSD 中的多个通道可以并发访问。固定内存再提供 1.4 倍的吞吐量提升，通过 GPU DMA 绕过 CPU。流水线提供了最终 1.5 倍的吞吐量改进，有助于避免在每个存储层级上对所有数据进行同步。

我们在一个容器中运行 ServerlessLLM 以限制它可以使用的 CPU 核心数。我们发现使用 4 个 CPU 核心时，ServerlessLLM 可以实现最大带宽利用率。我们在批量读取中设置了足够大的块大小（16MB），以减少读取次数，并且基于固定内存的块池不需要额外的 CPU 周期来进行数据复制。

7.3 ServerlessLLM 模型调度器

在本节中，我们评估了 ServerlessLLM 的簇调度器在测试平台 (ii) 上的性能。我们将 ServerlessLLM 与两个调度器进行比较——事实上的无服务器调度器和 Shepherd [90] 调度器。无服务器调度器随机选择任何可用的

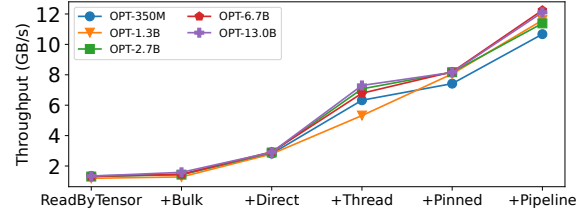


图 7: 检查点加载器的性能分解

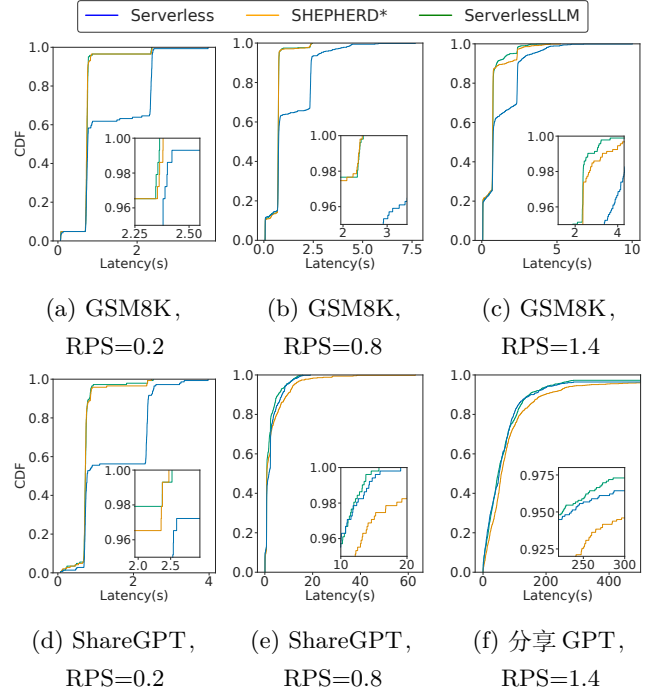


图 8: RPS 对模型加载调度器的影响。

GPU，并且不包含任何针对加载时间的优化。我们实现了 Shepherd 调度器，并使用 ServerlessLLM 的加载时间估计策略来识别正确的 GPU。我们将修改后的调度器称为 Shepherd*。因此，原则上，Shepherd* 和 ServerlessLLM 将选择相同的 GPU。然而，Shepherd* 将继续依赖抢占，而 ServerlessLLM 将依赖实时迁移以确保更低的延迟时间。

图 8a 显示了我们针对 OPT-6.7B 模型和 GSM8K 以及 ShareGPT 数据集运行所有三个调度器的场景结果，同时增加每秒请求数。ShareGPT 数据集的平均推理时间比 GSM8K 长 3.7 倍。图 8a 和图 8d 显示了两个数据集都没有局部性竞争的情况。由于 GPU 的随机分配，服务器端调度器无法利用基于局部性的调度，导致延迟更长。有 40% 的时间，模型需要从 SSD 加载，因为

GPU 是随机分配的。由于没有迁移或抢占, Shepherd 和 ServerlessLLM 的性能相似。

当调度器面临中等请求数时, 对于 GSM8K (图 8b, 无 locality-aware 调度), 加载时间开始导致队列延迟, Serverless 调度器的结果使 P99 延迟增加了 1.86 倍。由于没有迁移或抢占, Shepherd 和 ServerlessLLM 的性能相似。在 ShareGPT (图 8e) 具有更长的推理时间时, 我们观察到 Shepherd* 的 P99 延迟比 ServerlessLLM 高 2 倍, 这是由于抢占造成的。由于 ServerlessLLM 在存在局部性竞争时依赖于实时迁移, 尽管迁移次数较高 (513 个请求中有 114 次) 相比抢占次数 (513 个请求中有 40 次), ServerlessLLM 的表现仍优于其他调度器。

在通过将每秒请求数增加到 1.4 来进一步对系统施加压力时, 对于 GSM8K, 可以清楚地观察到实时迁移和抢占的影响。ServerlessLLM 在 P99 延迟方面分别比 Shepherd* 和 Serverless 调度器高出 1.27 倍和 1.95 倍。总共处理 925 个请求时, 分别发生 9 次抢占和 53 次迁移。如第 5.1 节所述, 与迁移相比, 抢占会导致更长的延迟。我们还观察到, 与 ServerlessLLM 相比, 使用 Shepherd* 时从 SSD 读取模型检查点的次数多出 2 倍。在 ShareGPT (图 8f) 中, 我们观察到 GPU 占用率达到 100%, 导致三种调度器都出现请求超时。³ Shepherd 相比 Serverless 和 ServerlessLLM 调度器表现最差, 即 P95 延迟分别高 1.43 倍和 1.5 倍。ServerlessLLM 和 Shepherd* 分别发出 64 次迁移和 166 次抢占, 共计处理 925 个请求。在此场景中, ServerlessLLM 的有效性受到资源限制的约束。

我们通过在 GSM8K 和 ShareGPT 数据集上运行更大的模型 (OPT-13B 和 OPT-30B) 进一步对系统施加压力。图 9 显示了这些实验的结果。对于较大的模型, 缓存到主内存中的局部性感知调度更为重要, 因为它可以带来更好的性能。由于 ServerlessLLM 和 Shepherd* 都具有局部性感知特性, 因此它们在调度请求时可以做出更好的决策, 从而获得更好的性能。而 Serverless 调度器随机做出决策, 在 GSM8K 的情况下, 我们观察到有 35-40% 的时间模型是从 SSD 加载的, 导致性能较差。我们在 ShareGPT 和 OPT-13B 实验中也看到了类似的行为。对于 OPT-30B 的 ShareGPT 案例, 模型大小为 66 GB, 因此在任何给定时间只能存储两个模型在主内存中, 这减少了局部性感知调度的影响。即使在这种极端情况下, ServerlessLLM 相比 Serverless 和 Shepherd*

³根据 OPT-6.7B 在 ShareGPT 数据集上的平均推理时间, 理论上最大 RPS 为 1.79。

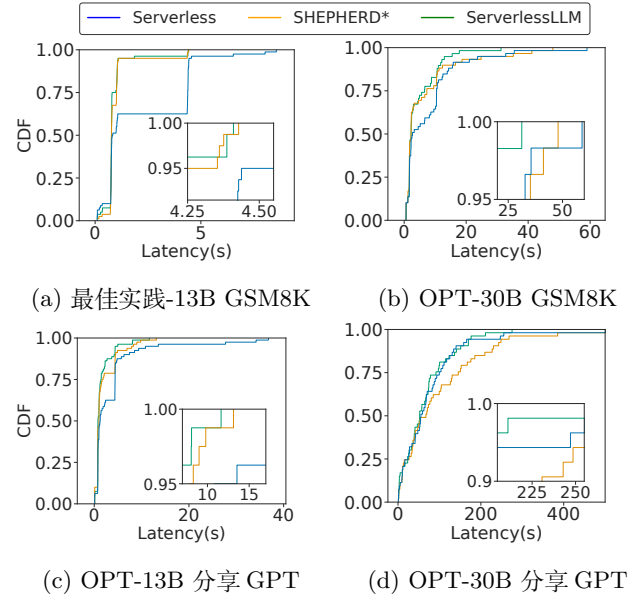


图 9: 数据集和模型对模型加载调度器的影响

分别实现了 P99 延迟降低 35% 和 45%。

时间估计. GPU 时间估计误差被限制在 5 毫秒以内, 而 SSD 加载误差被限制在 40 毫秒以内。然而, 我们确实观察到 CUDA 驱动程序调用的不稳定性。例如, 在迁移模型时, 我们注意到使用 `torch.cuda.empty_cache()` 清理 GPU 状态 (如 KV 缓存) 会导致估计不准确, 导致平均欠估计 25.78 毫秒。虽然不频繁, 但在 119 次迁移中, 有一次在清理 GPU 状态时观察到最大欠估计为 623 毫秒 (如图 8e 所示)。

7.4 整个 ServerlessLLM 在动作中

我们旨在将整个 ServerlessLLM 与测试平台 (ii) 上的无服务器工作负载一起部署。在这里, 我们将 ServerlessLLM 与最先进的分布式模型服务系统进行比较: (i) Ray Serve (版本 2.7.0), 我们对其进行了扩展以支持具有与最先进的无服务器解决方案 (如 KServe) 性能相匹配的无服务器推理场景; (ii) 带有缓存的 Ray Serve, 我们对其进行了改进以采用每个服务器上的本地 SSD 缓存 (使用如 ServerlessLLM 中的 LRU 策略) 以避免昂贵的模型下载; (iii) KServe (版本 0.10.2), 专为 Kubernetes 集群设计的最先进的无服务器推理系统。

对于最佳性能, Ray Serve 及其缓存变体都通过将模型检查点存储在本地 SSD 上, 并假设专用的 10 Gbps 网络来估算下载延迟而得到增强。对于每个系统, 我们

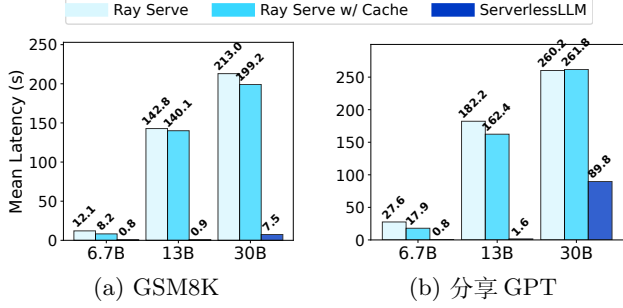


图 10: 数据集和模型对整体服务系统的影响。

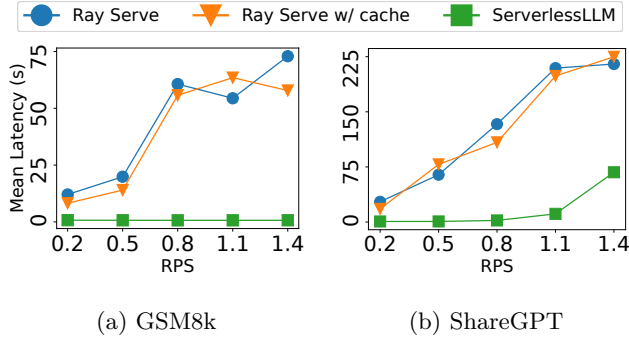


图 11: 可再生能源配额制对整体供电系统的影响。

按照先前的工作 [60] 将最大并发度设置为一，并将保活期设置为其加载延迟相等。我们启动并行的 LLM 推理客户端以生成各种工作负载，其中每个请求具有 300 秒的超时阈值。

加载优化检查点的有效性. 我们旨在评估在完整的无服务器工作负载中加载优化检查点的有效性，使用各种模型大小和数据集以多样地测试检查点加载程序。

在本实验中，如图 10 所示，Ray Serve 和带有缓存的 Ray Serve 都利用了 Safetensors。由于模型的大小较大，SSD 缓存无法容纳所有模型，因此需要从存储服务器下载部分模型。对于 OPT-6.7B 和 GSM 8K，ServerlessLLM 启动模型的平均时间为 0.8 秒，而 Ray Serve 需要 12.1 秒，带缓存的 Ray Serve 需要 8.2 秒，表现出超过 10 倍的改进。即使在网络更快的情况下（例如 100 Gbps），Ray Serve 的平均延迟可以降到 3.8 秒，但仍比 ServerlessLLM 慢 4.7 倍。随着模型规模的增大，模型加载器的重要性更加突出，因为 ServerlessLLM 可以利用并行 PCIe 链路，在多个 GPU 上从固定内存池加载分区的大模型。例如，对于 OPT-30B，ServerlessLLM 仍然可以在 7.5 秒内启动模型，而 Ray Serve 的时间上

升到 213 秒，带缓存的 Ray Serve 到 199.2 秒，显示出 28 倍的改进。

这一显著的延迟差异对 LLM 服务的用户体验产生了实质性影响。我们的观察表明，ServerlessLLM 在 OPT-30B 上可以在 300 秒超时内满足 89% 的请求，而带有缓存的 Ray Serve 仅能处理 26%。

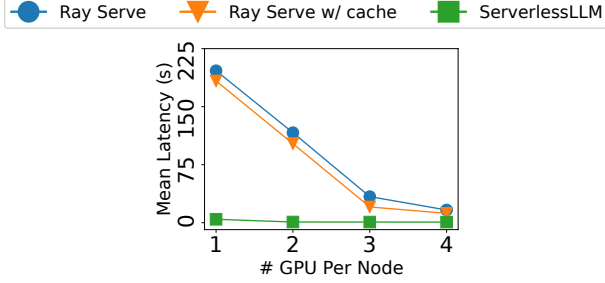
使用 ShareGPT 数据集（图 10b），其推理时间比 GSM 8K 长 3.7 倍，这对模型加载者来说变得更加严峻。对于 6.7B 和 13B 这样的模型，ServerlessLLM 的平均延迟分别为 0.8 秒和 1.6 秒，而 Ray Serve 和带有缓存的 Ray Serve 的延迟分别飙升至 182.2 秒和 162.4 秒。当使用 OPT-30B 时，ServerlessLLM 开始面临 GPU 资源限制（所有 GPU 都被占用且迁移无法释放更多资源），导致延迟增加到 89.9 秒。然而，这仍然显著优于带有缓存的 Ray Serve，后者延迟达到 261.8 秒。

LLM 实时迁移和加载调度器的有效性. 在评估 LLM 实时迁移和加载调度器的有效性时，我们创建了具有不同 RPS 级别的工作负载。高 RPS 场景突显了实现负载均衡和具备感知本地性的调度的重要性，因为仅仅加快模型加载速度不足以解决在高 RPS 级别下常见的资源争用问题。

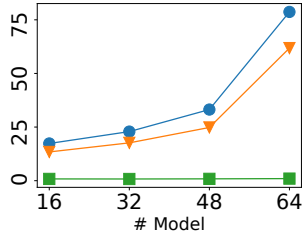
从图 11a 可以看出，配备 GSM8K 的 ServerlessLLM 即使在 RPS 增加时也始终维持大约 1 秒的低延迟。相比之下，Ray Serve 和带有 Cache 的 Ray Serve 在 RPS 超过 0.5 时经历上升的延迟，这可以归因于 GPU 资源短缺。与 ServerlessLLM 不同，它们无法迁移 LLM 推理以释放本地性或实现负载均衡，从而导致性能下降。

随着更具挑战性的 ShareGPT 工作负载，如图 11b 所示，ServerlessLLM 在 RPS 从 0.2 到 1.1 的范围内相比 Ray Serve 和带有缓存的 Ray Serve 保持了显著的性能提升——最高可达 212 倍。然而，在 RPS 为 1.4 时，ServerlessLLM 的延迟开始上升，这表明尽管有动态迁移和优化的服务器调度，有限的 GPU 资源最终还是影响了 ServerlessLLM 的性能。

资源效率. ServerlessLLM 在低模型启动延迟方面的最大优势之一是在服务 LLM 时对资源的节省贡献。我们通过改变每台服务器上可用的 GPU 数量来表示不同的资源分配级别。如图 12a 所示，ServerlessLLM 对弹性资源具有良好的扩展性。即使每台服务器仅配备一个 GPU，ServerlessLLM 也可以通过高效的迁移和交换实现 4 秒的延迟。相比之下，带有缓存的 Ray Serve 每台



(a) 多结点每结点的 GPU 数量 impacts



(b) 影响的#模型

图 12: 系统可扩展性与资源效率

服务器至少需要四个 GPU 才能达到 12 秒的延迟，这仍然高于 ServerlessLLM 在每个结点仅使用一个 GPU 的表现。在更大的集群中，ServerlessLLM 的资源节约效率预计会变得更加明显，因为更大的集群提供了更多关于实时迁移和服务器调度的选择。

当在簇中保持固定数量的 GPU 并增加 LLM 的数量时，ServerlessLLM 的资源效率变得更加明显。在图 12b 中，由于模型数量有限，带有缓存的 Ray Serve 可以在延迟性能上与 ServerlessLLM 匹配。然而，随着模型数量的增长，性能差距逐渐扩大，展示了 ServerlessLLM 在大规模无服务器平台中的潜在适用性。

KServe 对比. 在我们的研究中，我们在 Kubernetes 簇中评估了 KServe 和 ServerlessLLM。鉴于我们的四服务器簇不适合 Kubernetes 部署，我们转而使用一台八 GPU 服务器，模拟四个结点，每个结点有两个 GPU。由于 KServe 的性能比我们在评估中考虑的其他基准慢，我们仅简要提及 KServe 的结果而不深入细节。

使用 KServe 时，GPU 结点最初的第一个词元延迟为 128 秒。此延迟主要是因为 KServe 通过 1 Gbps 网络从本地 S3 存储下载 OPT-6.7B 模型检查点花费了 114 秒。但是，在应用与 Ray Serve 相同的增强功能后，我们将第一个词元延迟减少到 28 秒。尽管有所改进，但

KServe 的最佳延迟仍然显著高于 ServerlessLLM 实现的延迟。值得注意的是，ServerlessLLM 是唯一能够将延迟降低到 1 秒以内的系统。

8 相关工作

无服务器推理系统. 大量研究集中在优化无服务器架构中的机器学习模型服务，目标是批量处理 [10,84,89]、调度 [60,87] 和资源效率 [24,43]。工业界解决方案如 AWS SageMaker 和 Azure ML [50]，以及开源的 KServe [16]，展示了实际实现。尽管取得了这些进展，但我们的论文表明，无服务器推理系统在 LLM 上的表现仍然不尽如人意。

无服务器冷启动的优化. 冷启动延迟是无服务器系统中的一个重要问题，通过多种策略来解决，包括快速镜像拉取 [79]、轻量级隔离 [45,53]、快照和恢复 [15,22,28,67,77]、资源预置 [64]、弹性资源配置 [48,78] 和 fork [9,80]。这些方法主要集中在减少容器或虚拟机的启动时间，而不加载大型外部状态。最近的研究探索了通过加快 GPU 与主机内存之间模型交换速度来优化冷启动 [39,88]，但 LLM 的可扩展性仍具有挑战性。相比之下，ServerlessLLM 通过针对 LLM 特定的创新有效减少了冷启动延迟，例如优化的检查点格式和加载流水线、实时迁移以及专为 LLM 推理特性设计的集群调度器。

在无服务器系统中利用局部性. 局部性在无服务器系统的各种优化策略中起着关键作用。这包括利用主机内存和本地存储进行数据缓存 [41,59,69]，优化共享日志的读取 [40]，以及提升无服务器有向非循环图 (DAG) 中的通信效率 [46,47]。与现有方法不同，ServerlessLLM 引入了针对 GPU 的高性能检查点缓存，显著提高了从多层本地存储加载检查点到 GPU 内存的速度。最近的研究 [8,86] 也认识到在编排无服务器函数时利用局部性的必要性。除了这些研究之外，ServerlessLLM 利用 LLM 特有的特性来改进基于局部性的服务器选择，并启动以局部性为导向的推理。

大型语言模型 (LLM) 服务系统. LLM 服务的近期进展改善了推理延迟和吞吐量。Orca [85] 在推理过程中使用连续批量处理以更好地利用 GPU 资源。AlpaServe [44] 表明模型并行可以提高吞吐量，同时满足服务水平目标 (SLO) 约束，尽管尚未在生成式模型上进行测试。vLLM [42] 引入了 PagedAttention 来高效管理 KV 缓

存。SplitWise [55] 通过将提示和令牌生成阶段分布在不同机器上来提升吞吐量。一些方法 [14, 66] 还使用存储设备从 GPU 上卸载参数，以管理大模型的规模。然而，这些系统通常忽视了模型加载的挑战，在多个模型共享 GPU 时会导致首词延迟增加。ServerlessLLM 通过专注于最小化加载延迟来补充这些吞吐量和延迟优化。

9 结论

本文介绍了 ServerlessLLM，一个专为 LLMs 设计的低延迟无服务器推理系统。ServerlessLLM 的设计揭示了系统研究的重大机遇，包括设计新的加载优化检查点，发现支持实时迁移的需求，当进行基于局部性的 LLM 推理时，并使无服务器集群调度器能够意识到检查点在集群中的局部性，以优化其模型调度决策。我们相信我们的工作可以扩展到确保集群中资源的公平性，并探索智能检查点放置的可能性。我们期待在未来解决这些问题。我们认为 ServerlessLLM 是释放无服务器计算对 LLMs 潜力的第一步。我们将继续开发 ServerlessLLM 的开源版本。鉴于其多功能性，我们希望它成为一个平台来测试新的研究想法。

10 致谢

我们真诚感谢我们的指导老师 Amar Phanishayee 以及 OSDI 审稿人给出的富有洞察力的反馈，这些反馈帮助提升了本文的质量。我们也感谢 Boris Grot 提出的宝贵意见。我们承认张昭楠、戴明宇和费玉森在本项目早期阶段所做的工作。我们对爱丁堡大学信息学院和爱丁堡国际数据设施提供的 GPU 资源表示感谢。本研究由英国工程与物理科学研究委员会 (UK EPSRC) 和腾讯公司资助。

参考文献

- [1] Amazon elasticache pricing. <https://aws.amazon.com/elasticache/pricing/?nc=sn&loc=5>. Accessed: 2024-05-31.
- [2] Custom LLMs. https://deepinfra.com/docs/advanced/custom_llms, 2024. Accessed on 2024-06-02.
- [3] Generative AI tailored to you. <https://fireworks.ai/>, 2024. Accessed on 2024-06-02.
- [4] The leading enterprise AI platform. <https://cohere.com/>, 2024. Accessed on 2024-06-02.
- [5] Run AI with an API. <https://replicate.com/>, 2024. Accessed on 2024-06-02.
- [6] Serverless endpoints for leading open-source models. <https://www.together.ai/products#inference>, 2024. Accessed on 2024-06-02.
- [7] Your data. your AI. your future. <https://www.databricks.com/>, 2024. Accessed on 2024-06-02.
- [8] Mania Abdi, Samuel Ginzburg, Xiayue Charles Lin, Jose Faleiro, Gohar Irfan Chaudhry, Inigo Goiri, Ricardo Bianchini, Daniel S Berger, and Rodrigo Fonseca. Palette load balancing: Locality hints for serverless functions. In Proceedings of the Eighteenth European Conference on Computer Systems, EuroSys '23, page 365 – 380, New York, NY, USA, 2023. Association for Computing Machinery.
- [9] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paarijaat Aditya, and Volker Hilt. SAND: Towards High-Performance serverless computing. In 2018 USENIX Annual Technical Conference (USENIX ATC 18), pages 923–935, Boston, MA, July 2018. USENIX Association.
- [10] Ahsan Ali, Riccardo Pincirol, Feng Yan, and Evgenia Smirni. Batch: machine learning inference serving on serverless platforms with adaptive batching. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–15. IEEE, 2020.
- [11] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Merouane Debbah, Etienne Goffinet, Daniel Hestlow, Julien Launay, Quentin Malartic, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. Falcon-40B: an open large language model with state-of-the-art performance. 2023.

- [12] Amazon. AWS S3. <https://aws.amazon.com/s3/>, 2023. Accessed on 2024-01-22.
- [13] Amazon. Serverless inference - minimizing cold starts. <https://docs.aws.amazon.com/sagemaker/latest/dg/serverless-endpoints.html>, 2023. Accessed on 2024-01-22.
- [14] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, SC '22. IEEE Press, 2022.
- [15] Lixiang Ao, George Porter, and Geoffrey M. Voelker. Faasnap: Faas made fast using snapshot-based vms. In Proceedings of the Seventeenth European Conference on Computer Systems, EuroSys '22, page 730 – 746, New York, NY, USA, 2022. Association for Computing Machinery.
- [16] The KServe Authors. Kserve. <https://github.com/kserve/kserve>, 2023. Accessed on 2024-01-22.
- [17] Jens Axboe. Flexible I/O Tester, 2022. Accessed on 2024-01-22.
- [18] Wei Bai, Shanim Sainul Abdeen, Ankit Agrawal, Krishan Kumar Attre, Paramvir Bahl, Ameya Bhagat, Gowri Bhaskara, Tanya Brokhman, Lei Cao, Ahmad Cheema, Rebecca Chow, Jeff Cohen, Mahmoud Elhaddad, Vivek Ette, Igal Figlin, Daniel Firestone, Mathew George, Ilya German, Lakhmeet Ghai, Eric Green, Albert Greenberg, Manish Gupta, Randy Haagens, Matthew Hendel, Ridwan Howlader, Neetha John, Julia Johnstone, Tom Jolly, Greg Kramer, David Kruse, Ankit Kumar, Erica Lan, Ivan Lee, Avi Levy, Marina Lipshteyn, Xin Liu, Chen Liu, Guohan Lu, Yuemin Lu, Xiakun Lu, Vadim Makhervaks, Ulad Malashanka, David A. Maltz, Ilias Marinos, Rohan Mehta, Sharda Murthi, Anup Namdhari, Aaron Ogus, Jitendra Padhye, Madhav Pandya, Douglas Phillips, Adrian Power, Suraj Puri, Shachar Raindel, Jordan Rhee, Anthony Russo, Maneesh Sah, Ali Sheriff, Chris Sparacino, Ashutosh Srivastava, Weixiang Sun, Nick Swanson, Fuhou Tian, Lukasz Tomczyk, Vamsi Vadlamuri, Alec Wolman, Ying Xie, Joyce Yom, Lihua Yuan, Yanzhao Zhang, and Brian Zill. Empowering azure storage with RDMA. In 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), pages 49–67, Boston, MA, April 2023. USENIX Association.
- [19] Anyscale Technical Blog. <https://www.anyscale.com/blog/loading-llama-2-70b-20x-faster-with-any-scale-endpoints>, 2023. Accessed on 2024-01-22.
- [20] Banana.dev Technical Blog. <https://www.banana.dev/blog/turboboot>, 2023. Accessed on 2024-01-22.
- [21] Microsoft Official Blog. Reinventing search with a new AI-powered Microsoft Bing and Edge, your copilot for the web. <https://blogs.microsoft.com/blog/2023/02/07/reinventing-search-with-a-new-ai-powered-microsoft-bing-and-edge-your-copilot-for-the-web/>, 2023. Accessed on 2024-01-22.
- [22] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. Seuss: skip redundant paths to make serverless fast. In Proceedings of the Fifteenth European Conference on Computer Systems, EuroSys '20, New York, NY, USA, 2020. Association for Computing Machinery.
- [23] Andrew A. Chien, Liuzixuan Lin, Hai Nguyen, Varsha Rao, Tristan Sharma, and Rajini Wijayawardana. Reducing the carbon impact of generative AI inference (today and in 2035). In HotCarbon, pages 11:1–11:7. ACM, 2023.
- [24] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. Serving Heterogeneous Machine Learning Models on Multi-GPU Servers with Spatio-Temporal Sharing. In 2022 USENIX Annual Technical Conference (USENIX

- ATC 22), pages 199–216, Carlsbad, CA, July 2022. USENIX Association.
- [25] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. CoRR, abs/2110.14168, 2021.
 - [26] GitHub Copilot. <https://github.com/features/copilot>, 2023. Accessed on 2024-01-22.
 - [27] DataBricks. LLM Inference Performance Engineering: Best Practices. <https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices>, 2023. Accessed on 2024-01-22.
 - [28] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, pages 467–481, 2020.
 - [29] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. J. Mach. Learn. Res., 23(1), jan 2022.
 - [30] Cloud Native Computing Foundation. etcd. <https://etcd.io>, 2023. Accessed on 2024-01-22.
 - [31] Yixiao Gao, Qiang Li, Lingbo Tang, Yongqing Xi, Pengcheng Zhang, Wenwen Peng, Bo Li, Yaohui Wu, Shaozong Liu, Lei Yan, et al. When cloud storage meets {rdma}. In 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21), pages 519–533, 2021.
 - [32] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>, 2022.
 - [33] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In OSDI, pages 443–462. USENIX Association, 2020.
 - [34] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685, 2021.
 - [35] HuggingFace. <https://huggingface.co>, 2023. Accessed on 2024-01-22.
 - [36] HuggingFace. Safetensors: ML Safer for All. <https://github.com/huggingface/safetensors>, 2023. Accessed on 2024-01-22.
 - [37] HuggingFace. Speed Comparison. <https://huggingface.co/docs/safetensors/speed>, 2023. Accessed on 2024-01-22.
 - [38] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. Zookeeper: Wait-free coordination for internet-scale systems. In Proceedings of the 2010 USENIX Conference on USENIX Annual Technical Conference, USENIXATC’10, page 11, USA, 2010. USENIX Association.
 - [39] Jinwoo Jeong, Seungsu Baek, and Jeongseob Ahn. Fast and efficient model serving using multi-gpus with direct-host-access. In EuroSys, pages 249–265. ACM, 2023.
 - [40] Zhipeng Jia and Emmett Witchel. Boki: Stateful serverless computing with shared logs. In Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, SOSP ’21, page 691 – 707, New York, NY, USA, 2021. Association for Computing Machinery.
 - [41] Ana Klimovic, Yawen Wang, Patrick Stuedi, Animesh Trivedi, Jonas Pfefferle, and Christos

- Kozyrak. Pocket: Elastic ephemeral storage for serverless analytics. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pages 427–444, Carlsbad, CA, October 2018. USENIX Association.
- [42] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In SOSP, pages 611–626. ACM, 2023.
- [43] Jie Li, Laiping Zhao, Yanan Yang, Kunlin Zhan, and Keqiu Li. Tetris: Memory-efficient serverless inference through tensor sharing. In 2022 USENIX Annual Technical Conference (USENIX ATC 22), 2022.
- [44] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In OSDI, pages 663–679. USENIX Association, 2023.
- [45] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. Rund: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing. In USENIX Annual Technical Conference, pages 53–68. USENIX Association, 2022.
- [46] Ashraf Mahgoub, Karthick Shankar, Subrata Mitra, Ana Klimovic, Somali Chaterji, and Saurabh Bagchi. SONIC: Application-aware data passing for chained serverless applications. In 2021 USENIX Annual Technical Conference (USENIX ATC 21), pages 285–301. USENIX Association, July 2021.
- [47] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), pages 303–320, Carlsbad, CA, July 2022. USENIX Association.
- [48] Luo Mai, Guo Li, Marcel Wagenländer, Konstantinos Fertakis, Andrei-Octavian Brabete, and Peter Pietzuch. {KungFu}: Making training in distributed machine learning adaptive. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), pages 937–954, 2020.
- [49] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.
- [50] Microsoft. Azure ML. <https://learn.microsoft.com/en-us/azure/machine-learning>, 2023. Accessed on 2024-01-22.
- [51] MinIO. MinIO. <https://min.io>, 2023. Accessed on 2024-01-22.
- [52] NVIDIA. NVIDIA DGX H100. <https://resources.nvidia.com/en-us-dgx-systems/ai-enterprise-dgx>, 2023. Accessed on 2024-01-22.
- [53] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. SOCK: Rapid task provisioning with Serverless-Optimized containers. In 2018 USENIX Annual Technical Conference (USENIX ATC 18), pages 57–70, Boston, MA, July 2018. USENIX Association.
- [54] OpenAI. ChatGPT: Optimizing Language Models for Dialogue. <https://openai.com/blog/chatgpt/>, 2022. Accessed on 2024-01-22.
- [55] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Riccardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting, 2023.

- [56] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5, 2023.
- [57] PyTorch. PyTorch Documentation: Saving and Loading Models. https://pytorch.org/tutorials/beginner/saving_loading_models.html, 2023. Accessed on 2024-01-22.
- [58] PyTorch. torch.load. <https://pytorch.org/docs/stable/generated/torch.load.html>, 2023. Accessed on 2024-01-22.
- [59] Francisco Romero, Gohar Irfan Chaudhry, Íñigo Goiri, Pragna Gopa, Paul Batum, Neeraja J. Yadwadkar, Rodrigo Fonseca, Christos Kozyrakis, and Ricardo Bianchini. FaaS[†]: A transparent auto-scaling cache for serverless applications. In *Proceedings of the ACM Symposium on Cloud Computing, SoCC '21*, page 122 – 137, New York, NY, USA, 2021. Association for Computing Machinery.
- [60] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. INFaaS: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 397–411, 2021.
- [61] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. Icebreaker: Warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 753–767, 2022.
- [62] ONNX Runtime. API Reference, `onnxruntime.backend.prepare`. https://onnxruntime.ai/docs/api/python/api_summary.html, 2023. Accessed on 2024-01-22.
- [63] AWS SageMaker. Machine Learning Service - Amazon SageMaker. <https://aws.amazon.com/pm/sagemaker/>, 2023.
- [64] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218. USENIX Association, July 2020.
- [65] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285*, 2023.
- [66] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116. PMLR, 23–29 Jul 2023.
- [67] Simon Shillaker and Peter Pietzuch. Faasm: Lightweight isolation for efficient stateful serverless computing. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 419–433. USENIX Association, July 2020.
- [68] Dharma Shukla, Muthian Sivathanu, Srinidhi Viswanatha, Bhargav Gulavani, Rimma Nehme, Amey Agrawal, Chen Chen, Nipun Kwatra, Ramachandran Ramjee, Pankaj Sharma, Atul Katiyar, Vipul Modi, Vaibhav Sharma, Abhishek Singh, Shreshth Singhal, Kaustubh Welankar, Lu Xun, Ravi Anupindi, Karthik Elangovan, Hasibur Rahman, Zhou Lin, Rahul Seetharaman, Cheng Xu, Eddie Ailijiang, Suresh Krishnappa, and Mark Russinovich. Singularity: Planet-scale, preemptive and elastic scheduling of AI workloads, 2022.

- [69] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Joseph E. Gonzalez, Joseph M. Hellerstein, and Alexey Tumanov. Cloud-burst: stateful functions-as-a-service. *Proc. VLDB Endow.*, 13(12):2438 – 2452, jul 2020.
- [70] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. Déjàvu: Kvcache streaming for fast, fault-tolerant generative LLM serving, 2024.
- [71] Mistral AI Team. Better, faster, stronger. *Mistral AI News*, April 2024. Accessed: 2024-05-31.
- [72] The Mosaic Research Team. Introducing DBRX: A new state-of-the-art open LLM. *Mosaic AI Research*, March 2024. Accessed: 2024-05-31.
- [73] The Ray Team. Ray serve. <https://docs.ray.io/en/latest/serve/index.html>, 2023. Accessed on 2024-01-22.
- [74] TensorFlow. TensorFlow Documentation: Using the Saved Model Format. https://www.tensorflow.org/guide/saved_model, 2023. Accessed on 2024-01-22.
- [75] TensorFlow. tf.saved_model.load. https://www.tensorflow.org/api_docs/python/tf/saved_model/load, 2023. Accessed on 2024-01-22.
- [76] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. *Llama 2: Open foundation and fine-tuned chat models*, 2023.
- [77] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '21*, page 559 – 572, New York, NY, USA, 2021. Association for Computing Machinery.
- [78] Marcel Wagenländer, Luo Mai, Guo Li, and Peter Pietzuch. Spotnik: Designing distributed machine learning for transient cloud resources. In *12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 20)*, 2020.
- [79] Ao Wang, Shuai Chang, Huangshi Tian, Hongqi Wang, Haoran Yang, Huiba Li, Rui Du, and Yue Cheng. FaaSNet: Scalable and fast provisioning of custom serverless container runtimes at alibaba cloud function compute. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 443–457. USENIX Association, July 2021.
- [80] Xingda Wei, Fangming Lu, Tianxia Wang, Jinyu Gu, Yuhang Yang, Rong Chen, and Haibo Chen. No provisioned concurrency: Fast RDMA-codesigned remote fork for serverless computing. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 497–517, Boston, MA, July 2023. USENIX Association.
- [81] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models, 2023.

- [82] xAI. Grok-1, 2024. Accessed: 2024-05-31.
- [83] Ming Xu. https://huggingface.co/datasets/shibin-g624/sharegpt_gpt4, 2023.
- [84] Yanan Yang, Laiping Zhao, Yiming Li, Huanyu Zhang, Jie Li, Mingyang Zhao, Xingzhen Chen, and Keqiu Li. Infless: a native serverless system for low-latency, high-throughput inference. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '22*, page 768 – 781, New York, NY, USA, 2022. Association for Computing Machinery.
- [85] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association.
- [86] Minchen Yu, Tingjia Cao, Wei Wang, and Ruichuan Chen. Following the data, not the function: Rethinking function orchestration in serverless computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1489–1504, Boston, MA, April 2023. USENIX Association.
- [87] Minchen Yu, Zhifeng Jiang, Hok Chun Ng, Wei Wang, Ruichuan Chen, and Bo Li. Gillis: Serving large neural networks in serverless functions with automatic model partitioning. In *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*, pages 138–148. IEEE, 2021.
- [88] Minchen Yu, Ao Wang, Dong Chen, Haoxuan Yu, Xiaonan Luo, Zhuohao Li, Wei Wang, Ruichuan Chen, Dapeng Nie, and Haoran Yang. FaaSvSwap: SLO-Aware, GPU-Efficient Serverless Inference via Model Swapping. *CoRR*, abs/2306.03622, 2023.
- [89] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. MArk: Exploiting cloud services for Cost-Effective, SLO-Aware machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 1049–1062, Renton, WA, July 2019. USENIX Association.
- [90] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. SHEPHERD: Serving DNNs in the Wild. In *NSDI*, pages 787–808. USENIX Association, 2023.
- [91] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona T. Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. OPT: Open Pre-trained Transformer Language Models. *CoRR*, abs/2205.01068, 2022.
- [92] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. Faster and cheaper serverless computing on harvested resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, SOSP '21*, page 724 – 739, New York, NY, USA, 2021. Association for Computing Machinery.